



Frameworks After Foundations — LangChain + LangGraph



For the last few weeks, we built planner, retrieval, ranking, validation, and judge layers in raw Python.

Today, frameworks stop looking magical and start looking understandable.

Goal: see exactly what LangChain and LangGraph are abstracting.

**First principles first.
Frameworks second.**

Frameworks are not new intelligence.
They are reusable packaging for
patterns we already built.



Why Frameworks Exist

As AI systems get bigger, we repeatedly need the same things:

- prompt templates
- model wrappers
- structured outputs
- retrievers and text splitters
- state passing and orchestration
- evaluation and tracing

**We can absolutely build all of this in plain Python.
In fact, we already did.**

**A framework should remove boilerplate,
not remove understanding.**

**A framework is not intelligence.
A framework is packaging.**



Is It Paid?

LangChain
Open-source
MIT license

LangGraph
Open-source
Free to use

LangSmith
Hosted tracing / evaluation /
observability
Paid platform layer

For learning LangChain and LangGraph, you do not need to buy anything.

The paid part in the ecosystem is mainly the hosted observability / tracing platform.



What We Built Ourselves vs Framework Abstractions

Our Raw Python System	Framework Idea
Prompt strings	PromptTemplate
OpenAI model calls	Chat model wrapper
Pydantic validation	Structured output parser
PDF loading + chunking	Document loader + text splitter
Embeddings + retrieval	Retriever abstraction
Orchestrator function	Chain / runnable / graph node
Judge layer	Evaluator chain / graph node

**LangChain is not inventing new patterns here.
It is naming and packaging patterns we already understand.**



LangChain — Basic Prompt + Model Pipeline

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import
    ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o-mini",
    temperature=0)

prompt = ChatPromptTemplate.from_template("""
    Analyze this job description:
    {job_description}
    """)

chain = prompt | llm

result = chain.invoke({
    "job_description": "We are hiring a Python
    Automation Engineer..."
})

print(result.content)
```

ChatOpenAI wraps the model call.

ChatPromptTemplate makes the prompt reusable.

prompt | llm creates a runnable pipeline.

invoke(...) runs the pipeline with input data.

Conceptually this is the same as our manual planner step — just expressed in framework language.



LangChain + Pydantic Structured Output

```
from typing import List
from pydantic import BaseModel

class ResumePlan(BaseModel):
    role_title: str
    must_have_skills: List[str]
    nice_to_have_skills: List[str]
    reason: str

structured_llm =
llm.with_structured_output(ResumePlan)
chain = prompt | structured_llm

plan = chain.invoke({
    "job_description": "We are hiring a Python
Automation Engineer..."
})

print(plan.must_have_skills)
```

Last week we manually cleaned model JSON and validated it.

Here the framework can connect model output directly to a Pydantic schema.

Same philosophy, less plumbing.

**JSON asks for shape.
Pydantic still enforces shape.**



Framework Retrieval Building Blocks

```
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import
RecursiveCharacterTextSplitter

embeddings = OpenAIEmbeddings(model="text-
embedding-3-small")

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(resume_text)
vector = embeddings.embed_query(chunks[0])
```

Text splitter = framework version of our `chunk_text(...)`.

OpenAIEmbeddings = framework wrapper over the same embedding call.

Architecture is unchanged. Interfaces are cleaner.

The framework is not changing the system idea.

It is changing the developer ergonomics.



Why LangGraph Exists

LangChain is useful for many linear or moderately structured workflows.

But our systems are no longer just linear.

Planner → Retrieval → Ranking → Shortlist →
Judge is already graph-shaped.

LangGraph core ideas

- shared state
 - nodes
 - edges
- conditional routing

**If LangChain feels like chaining steps,
LangGraph feels like drawing the workflow as a state machine.**



Our Week 5 System Was Already Graph-Shaped



```
state = {
    "job_description": ...,
    "plan": ...,
    "retrieved_chunks": ...,
    "candidate_rankings": ...,
    "shortlist": ...,
    "evaluation_report": ...
}
```

**In raw Python, we passed data step by step.
In LangGraph, shared state becomes the first-class object.**



LangGraph — Conceptual Node + Edge Snippet

```
from langgraph.graph import StateGraph, END

def planner_node(state): ...
def retrieval_node(state): ...
def shortlist_node(state): ...
def judge_node(state): ...

graph = StateGraph(HiringState)
graph.add_node("planner", planner_node)
graph.add_node("retrieval", retrieval_node)
graph.add_node("shortlist", shortlist_node)
graph.add_node("judge", judge_node)

graph.set_entry_point("planner")
graph.add_edge("planner", "retrieval")
graph.add_edge("retrieval", "shortlist")
graph.add_edge("shortlist", "judge")
graph.add_edge("judge", END)

app = graph.compile()
result = app.invoke({"job_description": "..."})
```

StateGraph(...) defines the shared workflow state.

Nodes are steps in the workflow.

Edges define flow between those steps.

compile() turns it into a runnable graph.

invoke(...) runs the graph with input state.

**Our Week 5 system was already graph-shaped.
LangGraph just makes that explicit.**



Raw Python vs Framework — When to Use What

Plain Python is excellent for:

- learning fundamentals
- small / medium systems
 - full control
 - custom logic
- easier early debugging

Frameworks help when you need:

- less boilerplate
- reusable components
- clearer orchestration
- shared team patterns
- graph/state workflows

**A framework should remove pain,
not add mystery.**



If I had shown you LangChain in Week 1, it might have looked like syntax.

Now it should look like architecture.

If I had shown you LangGraph in Week 1, it might have looked abstract.

Now it should look like a cleaner representation of systems you already built.

You already built what frameworks abstract.

Frameworks will now help you accelerate, not confuse you.